

NLP: Attention & Transformers

Session 4 — Complete Study Notes

Topics: LSTM Review · Seq2Seq · Attention Mechanism · Transformer Architecture

1. LSTM — Long Short-Term Memory (Recap)

The **Long Short-Term Memory** network (Hochreiter & Schmidhuber, 1997) was designed to solve the **vanishing gradient problem** that plagued vanilla RNNs on long sequences. The key innovation is maintaining two separate state vectors: the **cell state** $\mathbf{C}^{(t)}$ (long-term memory highway) and the **hidden state** $\mathbf{a}^{(t)}$ (short-term working memory). Four gating mechanisms learn what to remember, forget, and output at each time step.

Gate	Activation	Formula	Role
Forget G_F	Sigmoid $\rightarrow [0,1]$	$\text{Sigmoid}(w_{fa} \cdot \mathbf{a}^{(t-1)} + w_{fx} \cdot \mathbf{x}^t + b_f)$	How much of the previous cell state $\mathbf{C}^{(t-1)}$ to keep. 0 = erase, 1 = keep.
Update G_U	Sigmoid $\rightarrow [0,1]$	$\text{Sigmoid}(w_{ua} \cdot \mathbf{a}^{(t-1)} + w_{ux} \cdot \mathbf{x}^t + b_u)$	How much new candidate information to write into the cell state.
Candidate $\mathbf{C}^\blacksquare$	Tanh $\rightarrow [-1,1]$	$\text{tanh}(w_{ca} \cdot \mathbf{a}^{(t-1)} + w_{cx} \cdot \mathbf{x}^t + b_c)$	New candidate values proposed for the cell state update.
Output G_O	Sigmoid $\rightarrow [0,1]$	$\text{Sigmoid}(w_{oa} \cdot \mathbf{a}^{(t-1)} + w_{ox} \cdot \mathbf{x}^t + b_o)$	Which parts of the (tanh-squashed) cell state to expose as $\mathbf{a}^{(t)}$.

State Update Equations

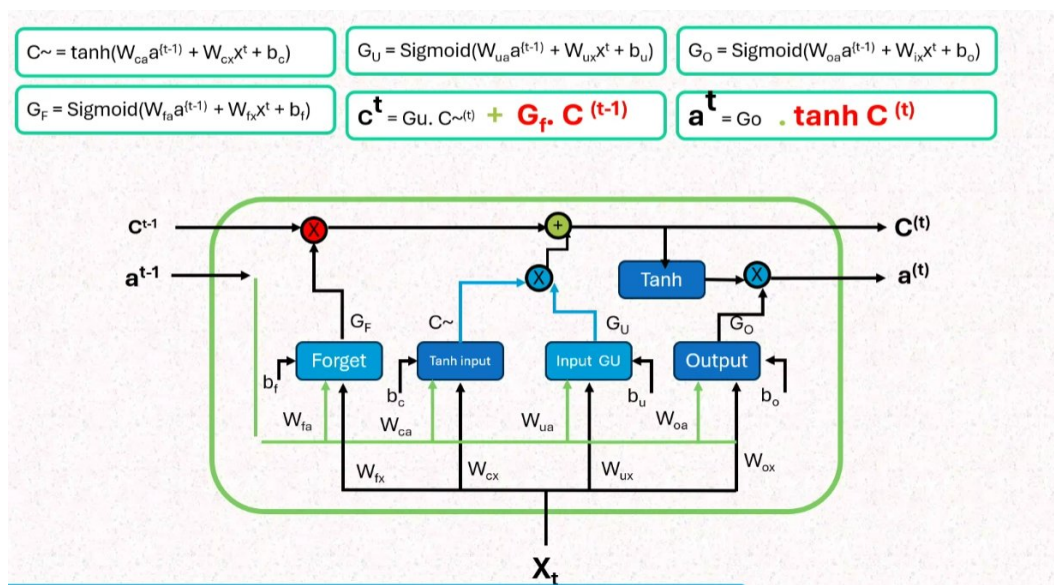
$$\mathbf{C}^{(t)} = G_U \cdot \mathbf{C}^\blacksquare^{(t)} + G_F \cdot \mathbf{C}^{(t-1)}$$

$$\mathbf{a}^{(t)} = G_O \cdot \text{tanh}(\mathbf{C}^{(t)})$$

Why does LSTM solve vanishing gradients?

The cell state $\mathbf{C}^{(t)}$ flows through the network with only *multiplicative* interactions (the forget gate). When $G_F \approx 1$ the gradient can flow back through many time steps without shrinking — the so-called **constant error carousel**. Vanilla RNNs force the gradient through a tanh at every step, squashing it exponentially.

Figure 1 — LSTM Cell Architecture



2. Encoder–Decoder (Sequence-to-Sequence with RNNs)

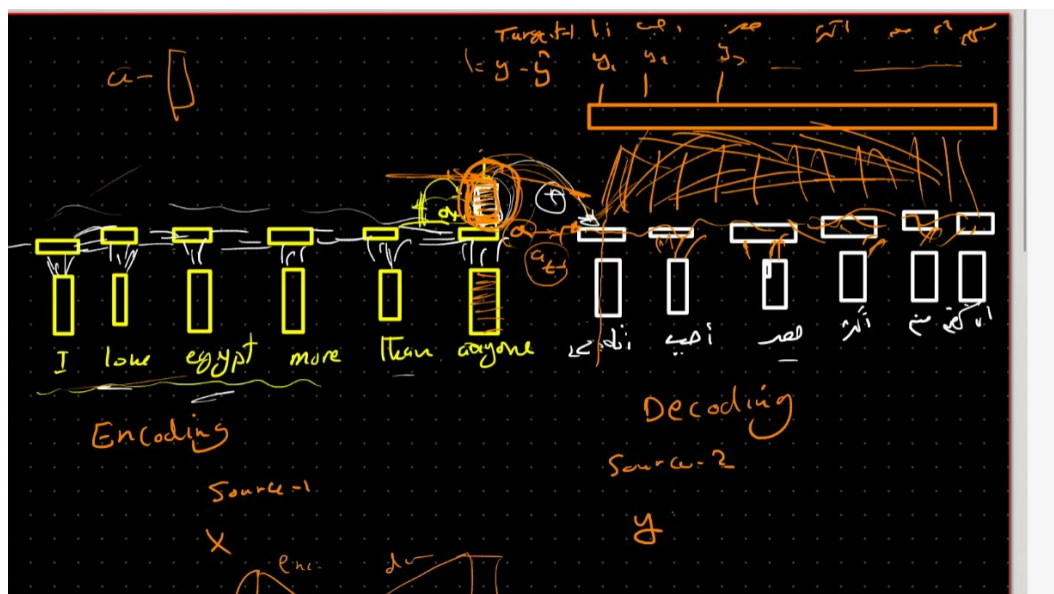
Before Transformers, machine translation used a **Seq2Seq** architecture (Sutskever et al., 2014) pairing two RNNs/LSTMs. The **encoder** reads the source sentence word-by-word and compresses the entire context into a single fixed-size **bottleneck vector** (the encoder's last hidden state). The **decoder** then uses that vector to auto-regressively produce target tokens ■.

Stage	What happens
Encoder	Source tokens $x_1 \dots x_T$ fed one-by-one into an RNN/LSTM. The final hidden state $h_T = \text{context vector}$ — a compressed summary of the whole input.
Bottleneck	The context vector is passed to the decoder as its initial hidden state. All source information must fit inside this single vector — a severe compression.
Decoder	An RNN/LSTM generates target tokens one at a time. At each step it receives the previous predicted token and the current hidden state to produce the next output via softmax.
Training	Teacher forcing: feed the ground-truth target token at each decoder step rather than the model's own prediction, which stabilises training.

■ Limitations of pure Seq2Seq (motivation for Attention)

- The bottleneck vector must store the entire source sentence — information is lost for long inputs.
- Sequential processing prevents parallelism; training on long sequences is slow.
- Later tokens in the encoder are over-represented in the context vector (recency bias).
- Bahdanau et al. (2015) introduced **Attention** to let the decoder selectively focus on different encoder states at each decoding step, bypassing the bottleneck.

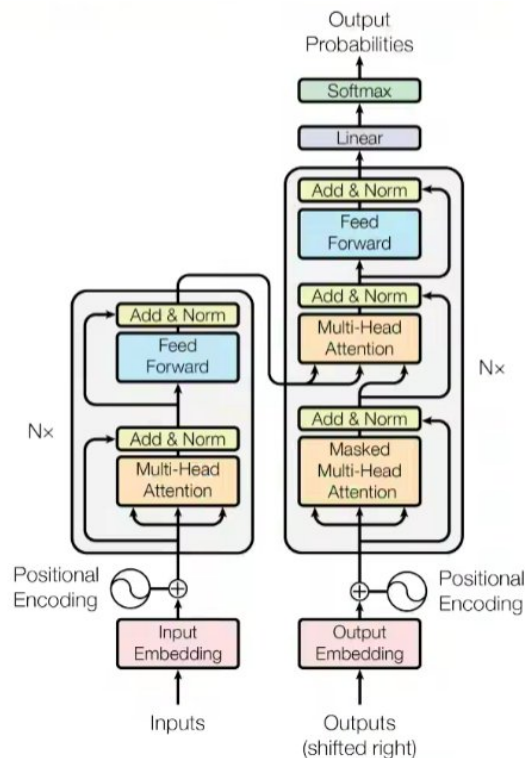
Figure 2 — Encoder–Decoder for Translation (e.g. English → Arabic)



3. The Transformer Architecture — "Attention Is All You Need"

Vaswani et al. (2017) published "**Attention Is All You Need**", introducing the Transformer — an architecture that discards recurrence entirely in favour of **self-attention**. This enables full parallelisation during training, captures global dependencies in a single layer, and scales far better than RNNs. The original model uses $d_{\text{model}}=512$, $h=8$ attention heads, $N=6$ encoder and decoder layers, and a feed-forward dimension of 2048.

Figure 3 — Full Transformer Architecture



3.1 Input Embedding

Each token in the vocabulary is mapped to a dense vector of dimension $d_{\text{model}}=512$ via a learned embedding matrix $E \in \mathbb{R}^{|V| \times 512}$. Tokens with similar meanings tend to cluster in this high-dimensional space. The same weight matrix is shared between the encoder input embedding, decoder input embedding, and the final linear projection (with a $\sqrt{d_{\text{model}}}$ scaling factor applied in the embedding layers).

3.2 Positional Encoding

Unlike RNNs, Transformers process all tokens *simultaneously*, so they have no inherent notion of word order. Positional encodings are added to the embeddings to inject sequence information. The original paper uses **fixed sinusoidal functions**:

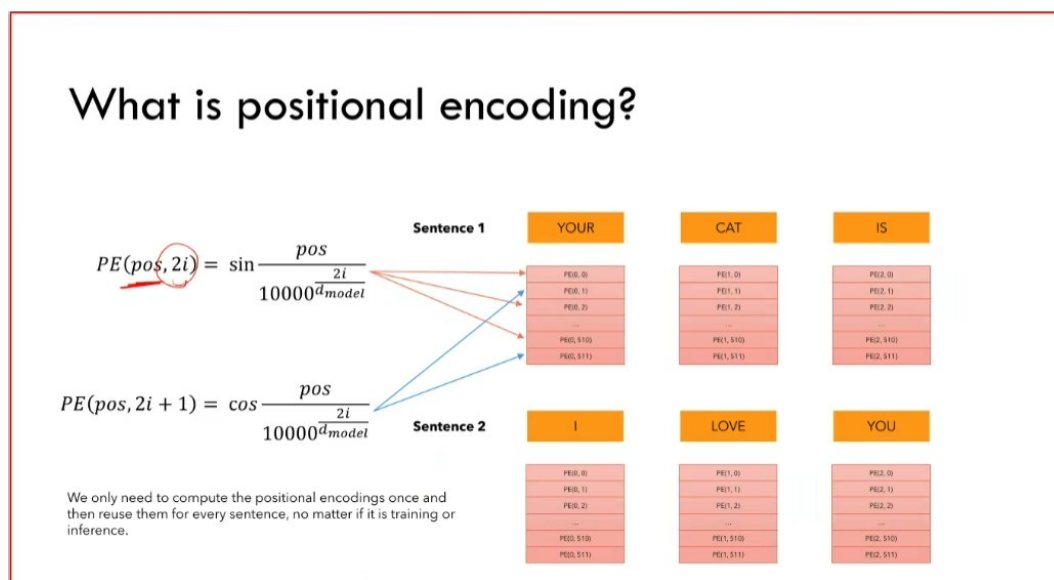
$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{2i / d_{\text{model}}}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{2i / d_{\text{model}}}) \end{aligned}$$

Why sinusoidal?

- Each position gets a unique encoding vector across all 512 dimensions.
- The sinusoidal pattern lets the model generalise to sequence lengths longer than those seen during training.

- $PE(pos+k)$ can be expressed as a linear function of $PE(pos)$ — the model can easily learn to attend by relative offset.
- Encodings are **fixed** (not learned) and computed once; reused for every batch at train and inference time.
- **Modern models** (RoPE, ALiBi) use learned or rotary positional encodings for better length generalisation.

Figure 4 — Positional Encoding: each word gets a 512-dim position vector



4. Self-Attention: Queries, Keys & Values (Q, K, V)

Self-attention allows every token to directly attend to every other token in one step, capturing long-range dependencies regardless of distance. It is the core computational primitive of the Transformer.

1. Linear Projection

The input matrix $X \in \mathbb{R}^{N \times d_{\text{model}}}$ is projected three times using learned weight matrices (no bias) to create:
 $Q = X \cdot W^Q$, $K = X \cdot W^K$, $V = X \cdot W^V$
where $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$. For a single head $d_k = d_{\text{model}}/h = 64$.

2. Similarity (Raw Scores)

Compute all pairwise query-key dot products: $\text{Score} = Q \cdot K^T$. Result is an $N \times N$ matrix — think of it as a covariance/correlation matrix between all token pairs. Entry (i,j) measures how relevant token j is when encoding token i .

3. Scaling

Divide by $\sqrt{d_k}$. Without this, large values of d_k push dot products into regions where the softmax gradient is near zero (saturation). Dividing by $\sqrt{d_k}$ keeps variance ≈ 1 , preventing softmax saturation.

4. Softmax (Attention Weights)

Apply softmax row-wise: each row of the $N \times N$ matrix becomes a probability distribution over all N positions. Entry (i,j) now represents 'what fraction of attention token i pays to token j '.

5. Weighted Aggregation

Multiply the attention weight matrix by V . Each output vector is a weighted sum of all value vectors — a context-enriched representation of each token.

Scaled Dot-Product Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Why no bias in the Q, K, V projections?

Adding a bias b to the projection shifts every token representation by a constant offset. This distorts the geometric structure of the embedding space that encodes meaning — two semantically distant tokens could appear closer simply because of the offset. Keeping projections bias-free preserves the inner-product geometry that attention relies on.

Intuition: What are Q, K, V?

Analogy	Meaning
query	What information is token i looking for?
key / index	What information does token j advertise/offer?
value / content	The actual information token j contributes if selected.

4.1 Multi-Head Attention

Rather than computing a single attention function, the model runs **$h=8$ independent attention heads** in parallel, each with its own learned projections $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$. Each head can specialise in a different type of relationship:

- Head 1 might learn syntactic subject–verb agreement.
- Head 2 might track coreference (pronouns $\rightarrow$ nouns).
- Head 3 might handle positional/local context.
- Other heads capture semantic similarity, negation scope, etc.

$$\text{head}_i = \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

The output projection $W^O \in \mathbb{R}^{h \cdot d_v \times d_{\text{model}}}$ combines all heads back to $d_{\text{model}}=512$. The final output shape is $N \times 512$ — same as the input, ready for the residual connection.

5. Add & Norm — Residual Connections + Layer Normalisation

After every sub-layer (attention or feed-forward) the Transformer applies:

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Residual / Skip Connection

The input x is added directly to the sub-layer's output before normalisation. This is identical to the residual connections in ResNet. Key benefits:

- Gradients flow directly from the loss to early layers without passing through many non-linearities — alleviates vanishing gradients in deep networks.
- If a sub-layer is harmful, the model can learn to zero out its weights; the skip connection then carries the original signal unchanged — effectively 'skipping' the block.
- Allows stacking $N=6$ (or many more) identical layers without degradation.

Layer Normalisation

Layer Norm (Ba et al., 2016) normalises across the *feature dimension* for each token independently (unlike Batch Norm which normalises across the batch dimension). For a vector x with d features:

$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

where μ and σ are the mean and std computed over $d=512$ features for that single token, and γ, β are learned scale/shift parameters. This stabilises the activations during training and prevents vanishing/exploding values.

Pre-LN vs Post-LN

The original paper uses Post-LN (norm after residual add). Modern models like GPT-2 and beyond use **Pre-LN** (norm before the sub-layer) which is more stable to train without learning rate warmup.

6. Position-wise Feed-Forward Network (FFN)

After self-attention has produced a *contextually enriched* representation of each token, a two-layer fully connected network is applied **identically and independently** to every position:

$$\text{FFN}(x) = \max(0, x \cdot w_1 + b_1) \cdot w_2 + b_2$$

Dimensions: $w_1 \in \mathbb{R}^{512 \times 2048}$, $w_2 \in \mathbb{R}^{2048 \times 512}$. The inner dimension 2048 is 4× the model dimension, providing a large capacity for non-linear feature extraction. The ReLU introduces non-linearity — without it the entire Transformer would collapse to a linear model.

Attention vs FFN — what does each do?

Self-attention is a *routing/mixing* operation: it re-represents each token as a weighted combination of other tokens' values — aggregating context but staying linear in V .

The FFN is the *computation/transformation* step: it applies a non-linear transformation to each token's already-contextualised representation, learning higher-level features. Research suggests the FFN layers act as key-value memory stores for factual knowledge.

7. Decoder Stack

The decoder has the same $N=6$ layers as the encoder but with **three** sub-layers per block instead of two:

Sub-layer	Description	Key Detail
Masked Multi-Head Self-Attention	Standard self-attention over the target sequence so far.	A causal mask sets all positions $j>i$ to $-\infty$ before softmax, ensuring token i can only attend to tokens $1\dots i$. Prevents the model from 'seeing the future' during training.
Cross-Attention (Encoder-Decoder Attention)	Q from decoder, K & V from encoder output.	This is the mechanism by which the decoder looks up relevant source-language information when generating each target token. Without it the decoder cannot access the input sentence.
Feed-Forward Add & Norm	Same 2-layer MLP as in the encoder.	Applied identically after each attention sub-layer.

Output Head: Linear → Softmax

The final decoder output ($N \times 512$) is projected by a linear layer to vocabulary size $|V|$ and passed through softmax to get a probability distribution over target tokens. The token with highest probability is selected as $\blacksquare^{(t)}$.

Inference (autoregressive generation)

- Step 1: Run the encoder once on the entire source sentence → encoder output matrix.
- Step 2: Start with the <SOS> (start-of-sentence) token as the first decoder input.
- Step 3: Decoder attends to its inputs so far + encoder output → predict next token.
- Step 4: Append predicted token; repeat from Step 3 until <EOS> (end-of-sentence) is produced.

■ The encoder is run only once. At each decoding step only the decoder is run, making inference efficient even for long outputs.

8. Training the Transformer

Aspect	Detail
Objective	Cross-entropy loss over all target positions simultaneously (teacher forcing: true targets fed as decoder input).
Optimiser	Adam with $\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-9}$.
LR Schedule	Warmup for 4000 steps then decay: $\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5})$.

Regularisation	Dropout ($p=0.1$) on attention weights, FFN outputs, and embedding sums. Label smoothing $\epsilon=0.1$.
Parallelism	Unlike RNNs, all positions are processed in parallel — major speedup. Training the original model took ~12 hrs on 8 P100 GPUs.
Complexity	Self-attention: $O(N^2 \cdot d)$ per layer. RNN: $O(N \cdot d^2)$. Transformers are cheaper for long sequences when $N < d$.

9. Architecture Summary Table

Component	Key Formula / Mechanism	Purpose
Token Embedding	Lookup in $E \in \mathbb{R}^{ V \times 512}$	Continuous token representation
Positional Encoding	$\sin/\cos$ of pos / $10000^{2i/d}$	Inject word-order information
Self-Attention	$\text{softmax}(QK^T/\sqrt{d_k}) \cdot V$	Global context aggregation in $O(1)$ layers
Multi-Head Attention	h parallel heads $\rightarrow$ Concat $\rightarrow W^O$	Capture multiple relationship types simultaneously
Add & Norm (Post)	$\text{LayerNorm}(x + \text{SubLayer}(x))$	Gradient flow + activation stability
FFN	$\text{ReLU}(xW_1 + b_1)W_2 + b_2$	Non-linear feature extraction; factual memory
Masked Self-Attention	Causal mask: set $j > i$ scores to $-\infty$	Prevent future-token leakage in decoder
Cross-Attention	$Q \leftarrow \text{decoder}, K/V \leftarrow \text{encoder output}$	Decoder reads source context
Linear + Softmax	$d_{\text{model}} \rightarrow V \rightarrow \text{probabilities}$	Output token selection

Key Equations Quick Reference

LSTM state update

$$c^{(t)} = g_u \cdot c^{(t-1)} + g_f \cdot c^{(t-1)} \mid a^{(t)} = g_o \cdot \tanh(c^{(t)})$$

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

Feed-Forward Layer

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

Add & Norm

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d}) \mid \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d})$$

10. Beyond the Original Transformer

The 2017 Transformer sparked a revolution. Key descendants worth knowing:

Model	Year	Innovation
BERT	2018	Encoder-only. Pre-trained with Masked LM + Next Sentence Prediction. Bidirectional context. Fine-tuned for classification, NER, QA.
GPT	2018	Decoder-only. Causal LM pre-training. Foundation of modern large language models (GPT-2, GPT-3, GPT-4, ChatGPT).
T5	2019	Encoder-decoder. Frames all NLP tasks as text-to-text. Trained on C4 corpus.
RoPE / ALiBi	2021-22	Improved positional encodings for extrapolation to longer sequences than seen in training.
Flash Attention	2022	IO-aware exact attention algorithm — same output, 2-4× faster, $O(N)$ memory instead of $O(N^2)$.
LLaMA / Mistral	2023	Open-weight LLMs using Pre-LN, RoPE, grouped-query attention (GQA), and SwiGLU activations.

★ Big Picture Takeaway

The Transformer's power comes from replacing sequential recurrence with **parallel self-attention**: every token directly interacts with every other token in $O(1)$ layers rather than $O(N)$ sequential steps. Combined with residual connections and layer norm enabling very deep networks, and multi-head attention capturing diverse relationship types, this architecture became the universal backbone of modern NLP — and has since been extended to vision (ViT), audio (Whisper), code (Codex), and multimodal models.